Chapter 39: Angular Material

1. Overview

Angular Material is the official component library maintained by the Angular team, built entirely on top of the **Component Dev Kit (CDK)** — a set of low-level, unstyled behavioral primitives (overlay, portal, a11y, layout, drag-drop, scrolling, bidi). Material components are essentially "CDK + Material Design visual styling + opinionated theming." Understanding this layering is the single most important mental model for the chapter: whenever an interviewer asks "how does X work internally," the answer almost always routes through a CDK primitive.

Since Angular Material v15+, the library moved to **Material Design 3 (M3)**, replacing the old `mat-core()` + `mat-theme()` Sass-only pipeline with a **system-tokens** architecture: a small set of Sass mixins emit **CSS custom properties** (`--mat-sys-*`, `--mdc-*`) at build time, and components consume those custom properties at runtime. This lets you retheme parts of the DOM tree at runtime (dark mode, per-section branding) without rebuilding Sass, and it is why modern "safe" style overrides use CSS variables instead of `::ng-deep`.

Material also ships **CDK-only, unstyled versions** of many components (`@angular/cdk/table`, `@angular/cdk/a11y`, `@angular/cdk/overlay`) so teams needing full visual control can build custom design systems on the same accessible, tested behavioral layer instead of fighting Material's CSS.

This chapter covers: the architecture (CDK vs Material), the M3 theming system, safe customization strategies, integrating custom controls into `<mat-form-field>` via `ControlValueAccessor` (and the newer `MatFormFieldControl` contract), accessibility guarantees, the interview-favorite trio (MatTable+sort+pagination, MatDialog, MatSnackBar), and tree-shaking / bundle-size concerns with standalone components.

---

2. Core Concepts

2.1 Architecture: CDK vs Material

- `@angular/cdk` — behavior without opinion. No CSS shipped for visuals (only structural/a11y CSS). Examples: `Overlay`, `Portal`, `FocusTrap`, `LiveAnnouncer`, `CdkTable`, `DragDropModule`, `CdkStepper`, `A11yModule`, `BidiModule`, `ScrollDispatcher`.
- `@angular/material` — CDK + Material Design look-and-feel + theming integration. `MatTable extends CdkTable`, `MatAutocomplete` positions itself with `CdkOverlay`, `MatDialog` wraps `CdkPortal` + `Overlay`, `MatSelect` uses `CdkOverlay` + `A11yModule` focus trapping, `MatSort` / `MatPaginator` are pure Material (data-source helpers, no overlay).
- Every Material component is a **thin visual/API layer** around CDK behavior. This is deliberate: CDK is the stable, low-churn foundation; Material's DOM/CSS can change between majors (this is the root cause of the "deep override breaks on upgrade" gotcha covered in §5).

2.2 Theming System

Legacy (pre-M3, Angular Material ≤14, "M2" themes)

```
@use '@angular/material' as mat;
@include mat.core();
$my-primary: mat.define-palette(mat.$indigo-palette);
$my-theme: mat.define-light-theme((color: (primary: $my-primary, ...)));
@include mat.all-component-themes($my-theme);
```

Everything was Sass maps consumed at compile time; there was no way to change theme at runtime without swapping stylesheets (or `class`-scoped duplicate themes, which bloated CSS).

M3 / system tokens (current)

```
@use '@angular/material' as mat;
html {
  @include mat.theme((
    color: mat.$violet-palette,
    typography: Roboto,
    density: 0,
  ));
}
```

`mat.theme()` emits a large block of **CSS custom properties** on the selector (`--mat-sys-primary`, `--mat-sys-on-surface`, `--mdc-filled-button-container-color`, etc.) derived from an M3 color palette (which itself is generated from a single seed color via HCT color space — tonal palettes at 13 tone stops: 0,10,20,...100). Components' internal Sass (compiled once into `@angular/material` package CSS) reference these custom properties via `var(--mat-sys-primary)`.

Key consequence: **theme = a runtime CSS custom property scope**, not a compile-time artifact. You can:

- Nest a different `mat.theme()` on a sub-tree (e.g., `.dark-panel { @include mat.theme((color: mat.$red-palette)); }`) and get correctly retheme components under it, with **no duplicated component CSS** (unlike M2 multi-theme).
- Override a single token directly: `.warn-btn { --mdc-filled-button-container-color: #b00020; }`.
- Toggle dark mode by swapping a *class* that redefines the custom-property values, not by re-including component mixins.

System-level vs component-level tokens: `--mat-sys-*` = M3 system tokens (color roles, typography scale, shape). `--mdc-*` = MDC (Material Design Components Web) component tokens that individual components read (button, checkbox, etc.). `--mat-*` (non-"sys") = Angular Material's own component-specific tokens for parts MDC doesn't cover (e.g., `--mat-table-*`).

You can theme a single component type without the whole system:

```
@include mat.button-theme($my-theme);       // legacy API, still supported
// or, M3 token-only override:
.my-toolbar { @include mat.toolbar-overrides((container-background-color: teal)); }
```

Every component exposes an `xxx-overrides()` mixin that lets you set only the tokens you care about — the recommended, future-proof customization surface.

2.3 Customizing Styles Safely

Ranked from safest/most-future-proof to riskiest:

1. **CSS custom property overrides** (`--mdc-*`, `--mat-*`) at a selector scope — works across encapsulation boundaries because custom properties inherit through the DOM regardless of `ViewEncapsulation`. This is the modern replacement for `::ng-deep`.
2. **Component `-overrides()` / `-theme()` Sass mixins** — official, versioned API; upgrades handle renames for you (with migration schematics).
3. `::part()` — for the handful of Material components that expose shadow-DOM-like parts (limited; Material doesn't use real Shadow DOM but some CDK-based overlays expose structural parts). Rare in practice.
4. **Global un-scoped stylesheet** targeting Material's own CSS classes (`.mat-mdc-button`) — works but is not "your" component's encapsulated style, so it's really just plain CSS cascade; stable-ish class names but layout/DOM structure can still shift.
5. `::ng-deep` (deprecated, will be removed) — pierces `ViewEncapsulation.Emulated` to style child component internals. Discouraged because: (a) Angular team has flagged for removal for years, (b) it still only works one direction (parent → descendant), (c) couples your CSS to Material's private DOM/class structure which is *not* a stable API and changes across minor/major versions.
6. `ViewEncapsulation.None` on your own component to leak styles globally — avoid; causes global collisions.

Rule of thumb for interviews: "Never reach into Material's internal DOM structure or private classes; theme via exposed Sass mixins or CSS custom properties, because those are the only parts of Material's styling surface covered by semver."

2.4 Form Field Integration — `ControlValueAccessor` and `MatFormFieldControl`

Two separate integration points, often conflated:

- `ControlValueAccessor` **(CVA)** — the Angular forms API contract (`writeValue`, `registerOnChange`, `registerOnTouched`, optionally `setDisabledState`) that lets *any* custom component participate in `formControl` / `ngModel` binding. This is framework-level, not Material-specific.
- `MatFormFieldControl<T>` — a Material-specific interface that lets a custom control be projected *inside* `<mat-form-field>` (so it gets the floating label, underline, prefix/suffix, error/hint slots, `mat-focus-indicator`, etc.). A control that implements `MatFormFieldControl` almost always *also* implements `ControlValueAccessor` so it can be driven by reactive forms, but the two are independent — you can have a CVA that isn't inside a `mat-form-field`, and (rarely) a form-field-control not wired to CVA.

`MatFormFieldControl<T>` contract highlights: `value`, `stateChanges: Observable<void>` (form field re-renders label/underline when this emits), `id`, `placeholder`, `ngControl: NgControl | null`, `focused`, `empty`, `shouldLabelFloat`, `required`, `disabled`, `errorState`, `controlType`, `setDescribedByIds(ids: string[])`, `onContainerClick(event)`. The form field looks up the control via `NG_VALUE_ACCESSOR`-style DI token `MatFormFieldControl` provided by your component.

2.5 Accessibility Built Into Material

Material bakes in a11y so teams don't reinvent it:

- **Focus management**: `CdkTrapFocus` / `FocusTrap` inside dialogs and menus; `FocusMonitor` distinguishes mouse/keyboard/programmatic focus origin (drives `.cdk-focused`, `.cdk-keyboard-focused` classes so focus rings only show for keyboard users).
- `LiveAnnouncer` (`@angular/cdk/a11y`) — pushes text into an `aria-live` region; used internally by `MatSort` (announces "Sorted by X ascending") and `MatSnackBar`.
- **ARIA roles/attributes wired automatically**: `MatSelect` → `role="listbox"` / `combobox` + `aria-expanded` / `aria-activedescendant`; `MatTable` → semantic `<table>` / `role="table"` (or native table elements); `MatDialog` → `role="dialog"`, `aria-modal="true"`, auto-focus first focusable element / the element configured via `autoFocus`, and returns focus to the triggering element on close.
- **High contrast mode support** (`cdk-high-contrast` mixins) emit extra CSS for Windows High Contrast Mode.
- **Keyboard interaction patterns** matching WAI-ARIA authoring practices are pre-built (arrow-key navigation in menus/selects/lists via `FocusKeyManager` / `ActiveDescendantKeyManager`).

This is a major interview differentiator vs "just build it with divs" — Material's value proposition is largely *tested accessibility for free*.

2.6 Tree-Shaking and Bundle Size

- Historically, `NgModule`-based Material required importing whole feature modules (`MatButtonModule`, `MatTableModule`) — each module only pulls in the code paths for the components it declares, so it *was* tree-shakeable at the module level (unused component modules were dropped), but importing `MatTableModule` for a `<mat-table>` also pulled in `MatSort` / `MatPaginator` "friend modules" if listed, even if unused directives were removed by the compiler when truly unreferenced in templates.
- Angular Material v15+ ships **standalone components** for every component (`MatButton`, `MatTableModule` still exists but you can `import { MatButton } from '@angular/material/button'` directly without the NgModule wrapper). Standalone imports give the best tree-shaking: only the directives/components actually referenced in a template are bundled, and Angular's build optimizer + Terser more reliably eliminate unused Ivy component factories when there's no NgModule `declarations` / `exports` array retaining them.
- CSS is **not** tree-shaken the same way if you use the monolithic theme mixins (`mat.all-component-themes($theme)` emits CSS for *every* component regardless of usage). Prefer per-component theme mixins (`mat.button-theme`, `mat.table-theme`) or the M3 `mat.theme()` (which is mostly custom-property declarations, cheap) plus only the density/typography you use, to avoid shipping CSS for unused components.
- Avoid importing the barrel `@angular/material` for TS symbols; always import from the specific entry point (`@angular/material/dialog`) — the package is structured as per-component secondary entry points specifically to support this.

---

3. Code Examples

3.1 MatTable with Sorting, Pagination, and Filtering (standalone)

```
// products-table.component.ts
import { Component, ViewChild, AfterViewInit } from '@angular/core';
import { MatTableModule, MatTableDataSource } from '@angular/material/table';
import { MatSortModule, MatSort } from '@angular/material/sort';
import { MatPaginatorModule, MatPaginator } from '@angular/material/paginator';
import { MatFormFieldModule } from '@angular/material/form-field';
import { MatInputModule } from '@angular/material/input';

export interface Product {
  id: number;
  name: string;
  category: string;
  price: number;
}

@Component({
  selector: 'app-products-table',
  standalone: true,
  imports: [
    MatTableModule,
    MatSortModule,
    MatPaginatorModule,
    MatFormFieldModule,
    MatInputModule,
  ],
  template: `
    <mat-form-field appearance="outline" class="filter-field">
      <mat-label>Filter</mat-label>
      <input matInput (keyup)="applyFilter($event)" placeholder="Search products" />
    </mat-form-field>

    <table mat-table [dataSource]="dataSource" matSort class="mat-elevation-z2">
      <ng-container matColumnDef="name">
        <th mat-header-cell *matHeaderCellDef mat-sort-header>Name</th>
        <td mat-cell *matCellDef="let row">{{ row.name }}</td>
      </ng-container>

      <ng-container matColumnDef="category">
        <th mat-header-cell *matHeaderCellDef mat-sort-header>Category</th>
        <td mat-cell *matCellDef="let row">{{ row.category }}</td>
      </ng-container>

      <ng-container matColumnDef="price">
        <th mat-header-cell *matHeaderCellDef mat-sort-header>Price</th>
        <td mat-cell *matCellDef="let row">{{ row.price | currency }}</td>
      </ng-container>

      <tr mat-header-row *matHeaderRowDef="displayedColumns"></tr>
```

```
      <tr mat-row *matRowDef="let row; columns: displayedColumns"></tr>

      <tr class="mat-row" *matNoDataRow>
        <td class="mat-cell" [attr.colspan]="displayedColumns.length">
          No products matching the filter.
        </td>
      </tr>
    </table>

    <mat-paginator [pageSizeOptions]="[5, 10, 25]" showFirstLastButtons></mat-paginator>
  `,
  styleUrl: './products-table.component.scss',
})
export class ProductsTableComponent implements AfterViewInit {
  displayedColumns = ['name', 'category', 'price'];
  dataSource = new MatTableDataSource<Product>([
    { id: 1, name: 'Widget', category: 'Hardware', price: 9.99 },
    { id: 2, name: 'Gadget', category: 'Electronics', price: 49.5 },
    // ...more rows
  ]);

  @ViewChild(MatSort) sort!: MatSort;
  @ViewChild(MatPaginator) paginator!: MatPaginator;

  ngAfterViewInit(): void {
    // Must be wired after view init because MatSort/MatPaginator
    // are only available once their template elements exist.
    this.dataSource.sort = this.sort;
    this.dataSource.paginator = this.paginator;
  }

  applyFilter(event: Event): void {
    const value = (event.target as HTMLInputElement).value.trim().toLowerCase();
    this.dataSource.filter = value;
    // Filtering resets pagination to page 0 automatically via dataSource.
    if (this.dataSource.paginator) {
      this.dataSource.paginator.firstPage();
    }
  }
}
```

```
// products-table.component.scss
.filter-field {
  width: 100%;
  max-width: 400px;
  margin-bottom: 1rem;
}

table {
  width: 100%;
}
```

```
// Safe override: component-level CSS custom property, no ::ng-deep needed.
:host {
  --mat-table-header-headline-color: var(--mat-sys-on-surface-variant);
}
```

3.2 MatDialog — Open and Return-Value Pattern

```
// confirm-dialog.component.ts
import { Component, inject } from '@angular/core';
import { MatDialogModule, MatDialogRef, MAT_DIALOG_DATA } from '@angular/material/dialog';
import { MatButtonModule } from '@angular/material/button';

export interface ConfirmDialogData {
  title: string;
  message: string;
}

@Component({
  selector: 'app-confirm-dialog',
  standalone: true,
  imports: [MatDialogModule, MatButtonModule],
  template: `
    <h2 mat-dialog-title>{{ data.title }}</h2>
    <mat-dialog-content>{{ data.message }}</mat-dialog-content>
    <mat-dialog-actions align="end">
      <button mat-button [mat-dialog-close]="false">Cancel</button>
      <button mat-flat-button color="warn" [mat-dialog-close]="true" cdkFocusInitial>
        Delete
      </button>
    </mat-dialog-actions>
  `,
})
export class ConfirmDialogComponent {
  dialogRef = inject(MatDialogRef<ConfirmDialogComponent, boolean>);
  data = inject<ConfirmDialogData>(MAT_DIALOG_DATA);
}
```

```
// caller.component.ts
import { Component, inject } from '@angular/core';
import { MatDialog } from '@angular/material/dialog';
import { ConfirmDialogComponent, ConfirmDialogData } from './confirm-dialog.component';
import { firstValueFrom } from 'rxjs';

@Component({ /* ... */ })
export class CallerComponent {
  private dialog = inject(MatDialog);

  async deleteItem(itemName: string): Promise<void> {
    const dialogRef = this.dialog.open<ConfirmDialogComponent, ConfirmDialogData, boolean>(
      ConfirmDialogComponent,
      {
        width: '360px',
```

```
        data: { title: 'Confirm delete', message: `Delete "${itemName}"? This cannot be
undone.` },
        autoFocus: 'dialog', // a11y: trap + move initial focus into the dialog
        restoreFocus: true,  // return focus to the trigger element on close
      }
    );

    // Preferred over subscribing to afterClosed() for a single result:
    const confirmed = await firstValueFrom(dialogRef.afterClosed());
    if (confirmed) {
      // proceed with deletion
    }
  }
}
```

3.3 Custom Form Control Implementing `ControlValueAccessor` (Material-styled field)

```
// phone-input.component.ts
import {
  Component, ElementRef, HostBinding, Input, OnDestroy, Optional, Self, inject,
} from '@angular/core';
import { ControlValueAccessor, NgControl, ReactiveFormsModule } from '@angular/forms';
import { MatFormFieldControl } from '@angular/material/form-field';
import { Subject } from 'rxjs';
import { FocusMonitor } from '@angular/cdk/a11y';

@Component({
  selector: 'app-phone-input',
  standalone: true,
  imports: [ReactiveFormsModule],
  template: `
    <div class="phone-input" [attr.aria-describedby]="describedBy">
      <input
        #input
        type="tel"
        [value]="value ?? ''"
        (input)="onInput(input.value)"
        (blur)="onTouched()"
        [disabled]="disabled"
      />
    </div>
  `,
  providers: [{ provide: MatFormFieldControl, useExisting: PhoneInputComponent }],
})
export class PhoneInputComponent implements ControlValueAccessor,
MatFormFieldControl<string>, OnDestroy {
  static nextId = 0;

  private fm = inject(FocusMonitor);
  private elementRef = inject(ElementRef<HTMLElement>);
```

```
  @HostBinding() id = `phone-input-${PhoneInputComponent.nextId++}`;
  stateChanges = new Subject<void>();

  value: string | null = null;
  focused = false;
  touched = false;

  @Input()
  get placeholder(): string { return this._placeholder; }
  set placeholder(value: string) { this._placeholder = value; this.stateChanges.next(); }
  private _placeholder = '';

  @Input()
  get required(): boolean { return this._required; }
  set required(value: boolean) { this._required = value; this.stateChanges.next(); }
  private _required = false;

  @Input()
  get disabled(): boolean { return this._disabled; }
  set disabled(value: boolean) { this._disabled = value; this.stateChanges.next(); }
  private _disabled = false;

  get empty(): boolean { return !this.value; }
  get shouldLabelFloat(): boolean { return this.focused || !this.empty; }

  controlType = 'app-phone-input';
  describedBy = '';
  setDescribedByIds(ids: string[]): void { this.describedBy = ids.join(' '); }
  onContainerClick(): void { this.elementRef.nativeElement.querySelector('input')?.focus();
}

  get errorState(): boolean {
    return this.ngControl?.invalid && (this.ngControl?.touched ?? false) || false;
  }

  private onChange: (value: string | null) => void = () => {};
  onTouched: () => void = () => {};

  constructor(@Optional() @Self() public ngControl: NgControl | null) {
    if (this.ngControl) {
      this.ngControl.valueAccessor = this;
    }
    this.fm.monitor(this.elementRef, true).subscribe((origin) => {
      this.focused = !!origin;
      this.stateChanges.next();
    });
  }

  onInput(raw: string): void {
    this.value = this.formatPhone(raw);
    this.onChange(this.value);
    this.stateChanges.next();
  }
```

```
  private formatPhone(raw: string): string {
    const digits = raw.replace(/\D/g, '').slice(0, 10);
    if (digits.length < 4) return digits;
    if (digits.length < 7) return `(${digits.slice(0, 3)}) ${digits.slice(3)}`;
    return `(${digits.slice(0, 3)}) ${digits.slice(3, 6)}-${digits.slice(6)}`;
  }

  // --- ControlValueAccessor ---
  writeValue(value: string | null): void { this.value = value; }
  registerOnChange(fn: (value: string | null) => void): void { this.onChange = fn; }
  registerOnTouched(fn: () => void): void { this.onTouched = fn; }
  setDisabledState(isDisabled: boolean): void { this.disabled = isDisabled; }

  ngOnDestroy(): void {
    this.stateChanges.complete();
    this.fm.stopMonitoring(this.elementRef);
  }
}
```

```
<!-- usage inside a mat-form-field, gets label/underline/error slots for free -->
<mat-form-field appearance="outline">
  <mat-label>Phone number</mat-label>
  <app-phone-input formControlName="phone" required></app-phone-input>
  <mat-error *ngIf="form.get('phone')?.hasError('required')">Phone is required</mat-error>
</mat-form-field>
```

3.4 M3 Theme Definition with System Tokens

```
// styles.scss
@use '@angular/material' as mat;

html {
  color-scheme: light;
  @include mat.theme((
    color: (
      theme-type: light,
      primary: mat.$violet-palette,
      tertiary: mat.$blue-palette,
    ),
    typography: (
      brand-family: 'Inter, sans-serif',
      plain-family: 'Roboto, sans-serif',
    ),
    density: 0,
  ));
}

// Scoped dark theme, e.g. toggled via a `.dark-mode` class on <body>
.dark-mode {
  color-scheme: dark;
  @include mat.theme((
```

```
    color: (
      theme-type: dark,
      primary: mat.$violet-palette,
    ),
  ));
}

// Per-component token overrides (safe, versioned surface)
.danger-zone {
  @include mat.button-overrides((
    filled-container-color: var(--mat-sys-error),
    filled-label-text-color: var(--mat-sys-on-error),
  ));
}

// Direct CSS custom property override for a one-off — also safe.
.compact-toolbar {
  --mat-toolbar-standard-height: 48px;
}
```

---

4. Internal Working

4.1 CDK Overlay as the Foundation for Floating UI

`MatAutocomplete`, `MatSelect`, `MatMenu`, `MatTooltip`, and `MatDialog` all render their floating content through `@angular/cdk/overlay`:

1. `Overlay.create(config)` creates an `OverlayRef` — a full-screen, absolutely-positioned container appended to a dedicated `<div class="cdk-overlay-container">` at the end of `<body>` (outside the app's component tree, so it escapes `overflow: hidden` / `z-index` stacking contexts of ancestors).
2. A `PositionStrategy` (commonly `FlexibleConnectedPositionStrategy`) computes where the overlay should sit relative to an origin element (e.g., the input for autocomplete), with fallback positions if the preferred one would overflow the viewport.
3. A `ScrollStrategy` decides what happens on ancestor scroll (`reposition()`, `close()`, `block()`, or `noop()`).
4. Content is projected into the `OverlayRef` via a `Portal` — either a `ComponentPortal` (instantiate a component into the overlay's injector/view) or `TemplatePortal` (attach an `ng-template`). `MatAutocomplete`'s panel is a `TemplatePortal`; `MatDialog`'s content is typically a `ComponentPortal`.
5. The overlay also composes a `BackdropStrategy` (click-to-close backdrop) and integrates `FocusTrap` / `InteractivityChecker` from `@angular/cdk/a11y` for keyboard containment.

So "MatAutocomplete uses CdkOverlay" concretely means: `MatAutocompleteTrigger` (the directive on the `matAutocomplete`-bound input) calls `overlay.create()`, builds a `FlexibleConnectedPositionStrategy` anchored to the input, attaches a `TemplatePortal` wrapping the `<mat-autocomplete>` panel template, and manages open/close state plus keyboard navigation (`ActiveDescendantKeyManager`) — none of that positioning/portal machinery is reimplemented by Material; it's 100% delegated to CDK.

4.2 How MatDialog Uses Portal + Overlay

`MatDialog.open(Component, config)`:

1. Creates an `OverlayRef` via `Overlay.create()` with a `GlobalPositionStrategy` (centers by default) and a backdrop (`hasBackdrop: true` by default, with `.cdk-overlay-dark-backdrop` styling).
2. Wraps your component in a `ComponentPortal`, attaches it to the `OverlayRef`, which instantiates your component with a **child injector** that provides `MatDialogRef` and `MAT_DIALOG_DATA` — this is how `inject(MatDialogRef)` / `inject(MAT_DIALOG_DATA)` resolve inside the dialog component without you wiring providers manually.
3. Wraps the instantiated component in an internal `MatDialogContainer` (itself a component) that adds `role="dialog"`, `aria-modal`, applies the `FocusTrap`, saves `document.activeElement` before opening, and — on close — calls `.focus()` back on it (`restoreFocus`).
4. `MatDialogRef` exposes `afterOpened()`, `afterClosed()`, `beforeClosed()`, `close(result)` as RxJS `Observable`/`Subject`s layered over the `OverlayRef`'s own `attachments()`/`detachments()` streams.
5. Closing calls `overlayRef.dispose()`, which detaches the portal (destroying the component and its view) and removes the DOM node from `cdk-overlay-container`.

Because the dialog's component tree is instantiated by the overlay's injector (not a structural directive in your template), it survives independent of the opening component's change-detection/view lifecycle — which is exactly why a dialog left open when its opener is destroyed does *not* automatically close (see §5).

4.3 How the Theming System Generates CSS

- At **library build time**, Angular Material's own Sass is compiled with each component's styles written in terms of `var(--mdc-component-token, fallback)` / `var(--mat-sys-token)`. This compiled CSS ships inside the npm package (`@angular/material/prebuilt-themes/*.css` for legacy, or plain component CSS referencing custom properties for M3) and does not change based on your theme.
- At **your app's build time**, your `@include mat.theme(...)` call runs Sass functions that: (a) take your seed/palette colors, (b) run them through the **HCT (Hue-Chroma-Tone) color space** algorithm from Material Color Utilities to generate tonal palettes and semantic color roles (primary/on-primary/primary-container/surface/error/etc.), (c) emit those as a flat block of `:root`/selector-scoped CSS custom property declarations (`--mat-sys-primary: #6750a4; ...`).
- At **runtime**, the browser cascade resolves `var(--mat-sys-primary)` inside the already-compiled component CSS against whatever scope's custom properties are nearest in the DOM tree — this is why nesting a second `mat.theme()` under a class works: CSS custom property scoping is just normal cascade/inheritance, no Angular involvement needed.
- Overrides (`mat.button-overrides(...)`) work the same way — they emit the same custom property names (`--mdc-filled-button-container-color`) at whatever selector you write the mixin under, so your override simply wins the cascade at that scope.

This design is why M3 theming is dramatically cheaper than M2 for "multiple themes on one page": M2 had to literally duplicate the full compiled CSS for each themed scope; M3 duplicates only a small property list.

---

5. Edge Cases & Gotchas

1. Deep style overrides breaking on version upgrades. Any selector targeting Material's internal DOM/class structure (`.mat-mdc-form-field-flex`, `::ng-deep .mat-select-panel`) is coupling to an *implementation detail*, not a public API. Material migrated its internals from the legacy custom renderer to **MDC Web** (Angular Material 15) and changed huge swaths of internal class names/DOM nesting (`mat-form-field` → `mat-mdc-form-field`) — every app using `::ng-deep`/private-class overrides broke and required rewriting. Rule: treat anything not exposed via a Sass mixin, a documented CSS custom property, or a documented `@Input`, as unstable.

2. Bundle size from importing whole NgModules instead of standalone components. `imports: [MatTableModule, MatSortModule, MatPaginatorModule, ...]` for a full app can pull in far more than needed if you import "convenience" umbrella modules or import from the root `@angular/material` package. Always import from the specific secondary entry point and, in standalone apps, import only the individual component classes actually used in the template — this both shrinks the bundle and makes it obvious in code review exactly what UI surface a component depends on.

3. `ViewEncapsulation` interactions with Material internals. `ViewEncapsulation.Emulated` (the default) adds `_ngcontent-*` attribute selectors to your component's own styles but Material's internal styles are compiled separately and are not encapsulated by *your* component's attribute — this is precisely why `::ng-deep` was ever "needed" (to pierce your own emulated encapsulation boundary so a selector could reach descendant Material DOM). Setting `ViewEncapsulation.None` on a component that hosts Material controls leaks all your component's own CSS globally with no scoping, which can unintentionally clash with Material's global classes. `ViewEncapsulation.ShadowDom` is effectively incompatible with older custom overrides and with CDK Overlay content (which is appended to `cdk-overlay-container` far outside your component's shadow root, so shadow-scoped styles won't reach it at all) — a real interview trap: "if you use Shadow DOM encapsulation, how do you style a MatDialog's content?" Answer: you can't via component-scoped shadow styles; you must use global styles, CSS custom properties, or `::part`-style APIs where available.

4. MatDialog memory leaks from unclosed dialogs. `MatDialogRef.afterClosed()` subscriptions in the *opening* component are not automatically torn down when the opener is destroyed if the dialog is still open — the dialog's `ComponentPortal` was instantiated in the *overlay's* injector, independent from the opener's view, so navigating away without closing the dialog leaves it alive in the `cdk-overlay-container`, still holding change-detection and DOM references (classic leak: forgetting to call `dialogRef.close()` in `ngOnDestroy`, or subscribing to `afterClosed()` without unsubscribing when the opener itself is destroyed first). Mitigation: track open dialogs and close them in `ngOnDestroy`, or use `MatDialog.closeAll()` on route change (default Material behavior since v9 auto-closes dialogs on `NavigationStart` **if** the dialog's `Overlay` is provided at root and `Router` events aren't disabled — but relying on implicit behavior is fragile; explicit `dialogRef.close()` is preferred).

5. `MatTableDataSource` sort/filter/pagination wiring order. Assigning `dataSource.sort`/`dataSource.paginator` must happen in `ngAfterViewInit` (not `ngOnInit`) because `@ViewChild` references to `MatSort`/`MatPaginator` are undefined until the view is initialized. Also, setting a custom `filterPredicate` replaces the default (which only checks a flattened string of all fields) and must be set before `.filter` is assigned to take effect on that first filter pass; and applying a new `.filter` value resets `paginator.pageIndex` only if you call `paginator.firstPage()` yourself — Material does **not** do this automatically, so users can end up on an empty "page 3" after filtering.

6. `MatFormFieldControl` `stateChanges` **forgetting to emit.** If a custom control mutates `focused` / `empty` / `errorState`-affecting fields without pushing to `stateChanges`, the floating label/underline/error UI silently goes stale (looks "broken" but no error is thrown) — a subtle, hard-to-diagnose bug specific to custom Material-integrated controls.

7. Prebuilt vs custom theme CSS duplication. Including both a prebuilt theme CSS file *and* your own `mat.theme()` / `all-component-themes()` call doubles compiled CSS shipped to the browser — common accidental bloat when migrating tutorials copy-paste both.

---

6. Interview Questions & Answers

Q1. What is the relationship between Angular Material and the Angular CDK?
A: The CDK is a set of unstyled, low-level behavioral primitives (overlay, portal, a11y, drag-drop, scrolling, table). Angular Material is built on top of the CDK, adding Material Design visuals and theming. Many Material components are thin wrappers: `MatTable extends CdkTable`, `MatAutocomplete` / `MatSelect` / `MatMenu` / `MatDialog` all use `CdkOverlay` + `CdkPortal` for their floating/overlay behavior. You can use the CDK alone to build a fully custom design system with Material's tested accessibility and interaction patterns but none of its visual opinions.

Q2. How does Material's theming system work in the current (M3) version, and how is it different from the old Sass-map theming?
A: M3 theming uses a small set of Sass mixins (`mat.theme()`, per-component `-theme()` / `-overrides()` mixins) that emit CSS custom properties (`--mat-sys-*`, `--mdc-*`) at whatever selector scope you include them under. Compiled component CSS (shipped in the `@angular/material` package) references these variables via `var(...)`, so the actual visual values are resolved by the browser's CSS cascade at runtime. The legacy (M2) approach used Sass maps consumed entirely at compile time (`mat.define-palette`, `mat.define-light-theme`, `mat.all-component-themes($theme)`), producing static, non-runtime-configurable CSS, and required duplicating full component CSS per additional theme scope. M3's custom-property approach means multiple themes/scopes on one page share the same compiled component CSS and only the property values differ — much cheaper and enables runtime theme switching (e.g., dark mode class toggle) without recompiling Sass.
Interviewer intent: checks whether the candidate has kept up with the M3 migration (a common gap for people who learned Material pre-2023) and whether they understand *why* custom properties are cheaper than Sass-map duplication — a real architectural reason, not just trivia.

Q3. Why is `::ng-deep` discouraged, and what should be used instead?
A: `::ng-deep` pierces `ViewEncapsulation.Emulated` to let a component's styles affect descendant component DOM, which is otherwise blocked by Angular's attribute-based style scoping. It's discouraged because: it's deprecated/planned for removal, it only works downward (parent → descendant) not on siblings/ancestors, and — most importantly for Material specifically — it couples your CSS selectors to Material's *private* internal DOM structure and class names, which are not covered by semver and have changed significantly across major versions (e.g., the MDC migration in v15). The supported alternatives are: CSS custom property overrides (`--mdc-*`, `--mat-*`), the component-specific `-theme()` / `-overrides()` Sass mixins, or (for full control) building on the CDK-only unstyled version of the component.

Q4. Walk through what happens internally when you call `MatDialog.open()`.
A: `MatDialog.open()` creates a `CdkOverlayRef` (via `Overlay.create()`) configured with a `GlobalPositionStrategy` (default centering) and a backdrop. It wraps the dialog component class in a `ComponentPortal` and attaches it to the overlay, which instantiates the component using a child injector that Material populates with `MatDialogRef` and `MAT_DIALOG_DATA` providers — this is why you can `inject(MatDialogRef)` / `inject(MAT_DIALOG_DATA)` inside the dialog without manual wiring. The instantiated component is nested inside an internal `MatDialogContainer` that adds `role="dialog"`, `aria-modal="true"`, traps focus via CDK's `FocusTrap`, remembers the previously focused element, and restores focus to it on close. `MatDialogRef` exposes `afterOpened()` / `afterClosed()` / `beforeClosed()` observables layered over the overlay's attach/detach streams; `close(result)` triggers overlay disposal, which detaches the portal (destroying the component) and removes the DOM node from the overlay container.

Q5. What's the difference between `ControlValueAccessor` and `MatFormFieldControl`?
A: `ControlValueAccessor` is Angular's framework-level contract (`writeValue`, `registerOnChange`, `registerOnTouched`, `setDisabledState`) enabling any component/directive to bind with `formControl` / `ngModel`. `MatFormFieldControl<T>` is Material-specific — it lets a custom control be hosted *inside* `<mat-form-field>` so it participates in the floating label, underline, prefix/suffix and `mat-error` / `mat-hint` rendering. A component that wants to look and behave like a native Material field typically implements both: CVA to hook into reactive/template-driven forms, and `MatFormFieldControl` (providing itself via `{ provide: MatFormFieldControl, useExisting: MyComponent }`) so `<mat-form-field>` can query its state (`focused`, `empty`, `errorState`, `shouldLabelFloat`) and react via the required `stateChanges: Observable<void>`.

Q6. Why must `dataSource.sort` and `dataSource.paginator` be assigned in `ngAfterViewInit` rather than `ngOnInit`?
A: `@ViewChild(MatSort)` / `@ViewChild(MatPaginator)` references are only populated once Angular has fully initialized the component's view, which happens after `ngOnInit` but before `ngAfterViewInit`. Assigning them earlier would attempt to read `undefined` view children.

Q7. How do you filter a `MatTableDataSource` and what's a common bug people introduce?
A: Set `dataSource.filter = someString.trim().toLowerCase()`; the default `filterPredicate` checks if the filter string is a substring of a flattened, lower-cased concatenation of all row field values. A common bug: forgetting that applying a new filter does not automatically reset `paginator.pageIndex`, so users can land on an out-of-range/empty page after filtering — you must call `dataSource.paginator.firstPage()` yourself after setting `.filter`. A second common bug: providing a custom `filterPredicate` after the first filter assignment, or forgetting it must return a boolean and receive `(data: T, filter: string)`.

Q8. Why does Material recommend importing components from their specific secondary entry points (e.g., `@angular/material/dialog`) rather than the root `@angular/material` package, and how does this interact with standalone components for tree-shaking?
A: Each Material component lives in its own secondary entry point specifically so bundlers only pull in the code for components actually imported; importing from the root barrel risks pulling in the whole public API surface re-exported there, defeating tree-shaking (depending on bundler/sideEffects config). Standalone components sharpen this further: without an `NgModule`'s `declarations` / `exports` arrays retaining references, only the exact component/directive classes referenced by a template are retained by the build optimizer, so an app using `MatButton` standalone and never touching `MatTable` should have zero table-related JS in its bundle. CSS is a separate concern — theme mixins like `mat.all-component-themes()` still emit CSS for every component regardless of usage, so bundle-size discipline requires pairing standalone/per-entry-point TS

imports with per-component (or M3 token-only) theme mixins.
Interviewer intent: tests whether the candidate conflates "tree-shakeable JS" with "tree-shakeable CSS" — a frequent gap, since Material's theming CSS is generated independently of which TS symbols you import.

Q9. A team upgraded Angular Material and their custom dark-mode override, written as `.dark-theme .mat-form-field-underline { background-color: white; }`, stopped working. Why, and how should it be fixed?
A: The class `.mat-form-field-underline` is part of Material's legacy (pre-MDC / pre-M3) internal DOM structure. Upgrading to the MDC-based Material 15+ (and further to M3 token architecture) renamed/restructured internals (e.g., `mat-mdc-form-field-*` classes, and the underline concept is now driven by MDC's own line-ripple/outline implementation controlled through `--mdc-*` / `--mat-*` custom properties). Because the selector targeted a private class rather than a documented API, it silently stopped matching anything. The fix is to replace it with the documented token override, e.g., `.dark-theme { --mdc-filled-text-field-active-indicator-color: white; }` (exact token name depends on `appearance`), or use `mat.form-field-overrides((...))`, which is guaranteed to keep working (or be migrated via schematics) across future versions.

Q10. How does `LiveAnnouncer` fit into Material's accessibility story, and can you give a concrete example of Material using it internally?
A: `LiveAnnouncer` (from `@angular/cdk/a11y`) programmatically injects text into a visually-hidden `aria-live="polite"` (or `"assertive"`) region so screen readers announce dynamic changes that wouldn't otherwise be noticed (since they don't involve a focus change). `MatSort` uses it internally to announce "Sorted by ascending/descending" whenever a user changes sort via `mat-sort-header`, since the visual sort-arrow change alone isn't perceivable to a screen-reader user. `MatSnackBar` similarly relies on a live region so a transient toast is announced even though it never receives focus.

Q11. Explain why a `MatDialog` can leak memory, and how you'd prevent it.
A: A dialog's component is instantiated via `ComponentPortal` inside the `Overlay`'s own injector/change-detection tree, independent of the component that opened it. If the opening component is destroyed (e.g., user navigates away) while the dialog is still open and its `afterClosed()` subscription hasn't fired/been cleaned up, the dialog (and everything it references, e.g., a large form or subscription chain) stays alive in the `cdk-overlay-container`, detached from the "logical" component tree that triggered it but still running change detection and holding memory. Prevention: explicitly call `dialogRef.close()` in the opener's `ngOnDestroy` for any dialogs it still owns, avoid capturing large closures/injectables unnecessarily in dialog data, and prefer `takeUntilDestroyed()`/manual unsubscribe on `afterClosed()` / `afterOpened()` subscriptions rather than assuming the dialog's lifecycle is tied to the opener's.
Interviewer intent: verifies the candidate understands that overlay-based content lives *outside* the normal parent-child destruction cascade — a subtlety that trips up developers who assume "component destroyed" implies "everything it opened is gone too."

Q12. If you needed pixel-perfect custom design system components but wanted Material's tested accessibility (focus trapping, keyboard navigation, ARIA), what would you use?
A: Build directly on the CDK rather than Material: `@angular/cdk/table` (`CdkTable`) for a fully unstyled data table with the same virtual-scroll/sticky-column/data-source machinery Material uses; `@angular/cdk/overlay` + `@angular/cdk/portal` for custom dropdowns/dialogs with the same positioning/backdrop/focus-trap behavior; `@angular/cdk/a11y` (`FocusTrap`, `FocusMonitor`, `LiveAnnouncer`, `ActiveDescendantKeyManager`) for keyboard/screen-reader behavior; `@angular/cdk/drag-drop`; `@angular/cdk/stepper`. This gives full visual control (your own CSS, no Material Design opinion) while reusing the same behavioral engine Material itself is built on, rather than fighting `::ng-deep`/token overrides to make Material look unlike Material.

Q13. What is `MatFormFieldControl.stateChanges` and why is it a common source of subtle bugs in custom controls?
A: `stateChanges` is an `Observable<void>` (backed by a `Subject`) that a `MatFormFieldControl` implementation must emit on whenever any state affecting the form field's rendering changes — focus state, emptiness, disabled state, error state, placeholder, required. `<mat-form-field>` subscribes to it to know when to re-run its own change detection for the floating label, underline color, and error/hint display. If a custom control mutates one of these properties (e.g., toggling `focused` inside a `FocusMonitor` callback) without calling `this.stateChanges.next()`, the form field's visual state silently desyncs from the actual control state — no runtime error is thrown, so it's typically caught only through manual QA (e.g., label not floating despite the field having a value, or an error message not appearing after a failed validation).

Q14. Does using `ViewEncapsulation.ShadowDom` on a component work well with Angular Material? What breaks?
A: Generally no, for two independent reasons. First, most of Material's own components are still built with `ViewEncapsulation.Emulated` (or none) internally and rely on global/ambient CSS custom properties for theming; wrapping your own component in real Shadow DOM changes how the CSS cascade reaches into it (custom properties still pierce shadow boundaries by design, so token overrides still work, but plain global stylesheet selectors do not). Second, and more seriously, CDK-overlay-based components (`MatDialog`, `MatMenu`, `MatSelect`, `MatAutocomplete`, tooltips) render their content into a `cdk-overlay-container` appended directly to `<body>`, entirely outside your component's shadow root — so styles scoped to your shadow root (or CSS custom properties set only inside it, since custom properties don't cross a shadow boundary outward from inside to a sibling subtree appended elsewhere in the document) will not reach that overlay content at all. In practice, teams using Shadow DOM encapsulation theme Material via document-level (not shadow-scoped) custom property declarations to make sure overlay content is covered.

---

7. Quick Revision Cheat Sheet

- **CDK vs Material**: CDK = unstyled behavior (`overlay`, `portal`, `a11y`, `table`, `drag-drop`); Material = CDK + Material Design visuals + theming. Most floating Material components (`MatSelect`, `MatAutocomplete`, `MatMenu`, `MatDialog`, tooltips) = `CdkOverlay` + `CdkPortal` under the hood.
- **Theming (M3)**: `mat.theme()` and per-component `-theme()` / `-overrides()` mixins emit CSS custom properties (`--mat-sys-*` system tokens, `--mdc-*` component tokens). Component CSS (compiled once, shipped in the package) consumes these via `var()`. Cascade/inheritance resolves the actual values at runtime — enables scoped/nested themes and runtime dark mode cheaply, unlike the old M2 Sass-map approach which duplicated full CSS per theme.
- **Safe customization order**: CSS custom property override > component `-overrides()` / `-theme()` mixin > global stylesheet on Material's own classes (risky but not encapsulation-piercing) > `::ng-deep` (deprecated, avoid) > `ViewEncapsulation.None` (avoid, global leakage).
- **Form integration**: `ControlValueAccessor` = framework-level forms binding contract (`writeValue` / `registerOnChange` / `registerOnTouched` / `setDisabledState`). `MatFormFieldControl<T>` = Material-specific contract to be hosted inside `<mat-form-field>` (`stateChanges`, `empty`, `focused`, `shouldLabelFloat`, `errorState`, `setDescribedByIds`). Usually implement both together; don't forget to call `stateChanges.next()` on every relevant mutation.
- **Accessibility for free**: `FocusTrap` / `FocusMonitor` (keyboard-only focus rings, dialog focus containment

+ restore), `LiveAnnouncer` (MatSort/MatSnackBar announcements), correct ARIA roles/keyboard patterns pre-built (`MatSelect` listbox/combobox, `MatDialog` `role="dialog"` / `aria-modal`).

- **MatTable+Sort+Paginator**: wire `dataSource.sort` / `dataSource.paginator` in `ngAfterViewInit` (ViewChildren not ready earlier); custom `filterPredicate` for non-default filtering; call `paginator.firstPage()` manually after filtering — it's not automatic.
- **MatDialog**: `dialog.open(Component, config)` returns `MatDialogRef<T, R>`; get the result via `dialogRef.afterClosed()` (or `firstValueFrom` for a promise); `[mat-dialog-close]="value"` sets the close result declaratively; overlay-hosted dialogs are not destroyed automatically when the opener is destroyed — close explicitly to avoid leaks.
- **MatSnackBar**: transient, non-modal, auto-dismiss (`duration`), announced via the same `LiveAnnouncer` mechanism; `snackBar.open(message, action, config)` returns a ref with an `onAction()` / `afterDismissed()` observable.
- **Tree-shaking**: import from specific secondary entry points (`@angular/material/button`, not the root barrel); prefer standalone component imports over NgModules for tightest JS tree-shaking; remember CSS from `mat.all-component-themes()` is *not* usage-gated — use per-component theme mixins or M3 token-only overrides to avoid shipping unused component CSS.
- **Version-upgrade risk**: anything selecting Material's private DOM/class names (not a documented mixin/token/`@Input`) is unstable across majors — the MDC migration (v15) is the canonical cautionary example.

Created By - Durgesh Singh